

AI _ NETWORK _ INTRUSION _ DETECTION

CHAPTER 1

INTRODUCTION

1.1 Introduction

In today's digital world, computer networks play a vital role in communication, business, education, banking, healthcare, and many other sectors. Every day, millions of users exchange information through the Internet. As the use of computer networks increases, the number of cyberattacks also continues to grow. Attackers attempt to gain unauthorized access to systems, steal sensitive information, spread malware, or disrupt network services. These malicious activities are known as network intrusions.

To protect computer networks from such attacks, organizations use Intrusion Detection Systems (IDS). An IDS continuously monitors network traffic and identifies suspicious activities that may indicate a cyberattack. Traditional intrusion detection systems mainly rely on predefined attack signatures. Although they are effective in detecting known attacks, they often fail to identify new or previously unseen attacks. As cyber threats continue to evolve, traditional methods alone are no longer sufficient.

Artificial Intelligence (AI) and Machine Learning (ML) have introduced new approaches to intrusion detection. Machine learning algorithms can analyze network traffic patterns and identify unusual activities automatically. Unlike traditional methods, machine learning-based intrusion detection systems can detect unknown attacks by learning the normal behavior of network traffic.

This project presents an AI-Based Network Intrusion Detection System using the Isolation Forest algorithm, an unsupervised machine learning technique. The proposed system uses the NSL-KDD dataset, which contains both normal and attack network traffic records. The dataset is preprocessed through data cleaning, label encoding, and feature scaling before training the machine learning model.

The trained model identifies abnormal network activities as anomalies without relying on attack labels during training.

The system also provides graphical visualizations such as prediction distribution, protocol distribution, confusion matrix, and network traffic analysis to help users understand the detection results. This project demonstrates how artificial intelligence can improve cybersecurity by detecting malicious activities efficiently and accurately.

1.2 Background

With the rapid growth of the Internet and cloud computing, organizations increasingly depend on computer networks to store and exchange critical information. This dependency has also attracted cybercriminals who attempt to exploit network vulnerabilities through various attacks such as Denial of Service (DoS), Probe attacks, User-to-Root (U2R) attacks, and Remote-to-Local (R2L) attacks.

Traditional security mechanisms like firewalls prevent unauthorized access but cannot detect every malicious activity occurring inside a network. Therefore, Intrusion Detection Systems (IDS) have become an essential component of modern cybersecurity.

Recent advancements in machine learning have significantly improved intrusion detection techniques. Machine learning models can learn network behavior and automatically identify suspicious traffic patterns. Among various machine learning techniques, Isolation Forest is particularly suitable for anomaly detection because it isolates unusual observations efficiently without requiring labeled training data.

1.3 Need for the Project

Cyberattacks are becoming increasingly sophisticated and difficult to detect using conventional security systems. Signature-based intrusion detection systems require frequent updates and cannot detect new attack patterns. Manual monitoring of large-scale network traffic is also time-consuming and impractical.

There is a growing need for an intelligent intrusion detection system capable of automatically identifying abnormal network behavior without prior knowledge of attack signatures. Machine learning provides an effective solution by analyzing network traffic patterns and detecting anomalies in real time.

The proposed system addresses these challenges by using an unsupervised machine learning algorithm to identify suspicious activities in network traffic. This reduces manual effort while improving the overall security of computer networks.

1.4 Problem Statement

Modern computer networks generate a massive amount of traffic every second, making manual monitoring impossible. Existing intrusion detection systems primarily depend on predefined attack signatures and often fail to identify unknown or zero-day attacks. This creates security vulnerabilities that can be exploited by attackers.

Therefore, there is a need to develop an intelligent intrusion detection system capable of automatically detecting abnormal network activities using machine learning techniques without relying entirely on labeled training data.

1.5 Objectives

The main objectives of this project are:

- To study the concept of network intrusion detection.
- To analyze network traffic using the NSL-KDD dataset.
- To preprocess the dataset by removing inconsistencies and encoding categorical features.
- To implement the Isolation Forest algorithm for anomaly detection.
- To classify network traffic as normal or suspicious.
- To evaluate the model using performance metrics.
- To visualize the prediction results using various graphs.
- To develop a simple and efficient AI-based intrusion detection system.

1.6 Scope of the Project

The scope of this project includes the application of machine learning techniques for identifying anomalous network traffic using the NSL-KDD dataset. The project demonstrates how unsupervised learning can detect suspicious activities without requiring labeled training data.

The developed model can be used for educational purposes, cybersecurity research, and understanding anomaly detection techniques. Although the project is implemented using an offline dataset, similar techniques can be extended to real-time network monitoring systems in future work.

1.7 Project Overview

The proposed intrusion detection system begins by loading the NSL-KDD dataset into Python using Jupyter Notebook. The dataset is then analyzed and preprocessed through duplicate removal, missing value verification, label encoding, and feature scaling.

The processed data is provided to the Isolation Forest algorithm, which identifies anomalous network activities by isolating unusual observations. The generated predictions are evaluated using a confusion matrix and classification report. Finally, various graphs including prediction distribution, protocol distribution, source bytes distribution, destination bytes distribution, and actual label distribution are generated to visualize the model's performance.

The complete implementation is carried out using Python, Pandas, NumPy, Matplotlib, and Scikit-learn.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

With the rapid growth of computer networks and Internet-based applications, cyber threats have become a major concern for individuals and organizations. Intrusion Detection Systems (IDS) are designed to monitor network traffic and identify unauthorized or malicious activities. Over the years, researchers have proposed various techniques ranging from traditional signature-based systems to modern machine learning approaches.

Traditional IDSs rely on predefined attack signatures and are effective only for known attacks. However, they often fail to detect new or unknown attacks. To overcome this limitation, researchers have explored artificial intelligence and machine learning techniques, which can learn patterns from data and identify anomalies automatically.

This chapter reviews some important studies related to intrusion detection and highlights the need for an AI-based anomaly detection system.

2.2 Traditional Intrusion Detection Systems

Traditional intrusion detection systems are generally classified into two categories:

Signature-Based Intrusion Detection

Signature-based IDS compares incoming network traffic with a database of known attack signatures. If a match is found, the activity is classified as malicious.

Advantages

- High accuracy for known attacks.
- Low false alarm rate for previously identified threats.

Disadvantages

- Cannot detect unknown or zero-day attacks.
- Requires frequent updates of attack signatures.
- Performance depends on the quality of the signature database.

Examples include Snort and Suricata.

Anomaly-Based Intrusion Detection

Anomaly-based IDS learns the normal behaviour of network traffic and detects activities that significantly deviate from this behaviour.

Advantages

- Can detect previously unseen attacks.
- Suitable for dynamic network environments.
- Does not rely entirely on predefined signatures.

Disadvantages

- May generate more false alarms.
- Requires proper feature selection and preprocessing.

Our project belongs to this category because it uses Isolation Forest, an anomaly detection algorithm.

2.3 Machine Learning in Intrusion Detection

Machine learning has significantly improved intrusion detection systems by enabling automatic analysis of network traffic. Instead of relying only on predefined rules, machine learning models recognize patterns and identify suspicious activities.

Machine learning techniques are generally divided into:

- Supervised Learning

Supervised learning uses labeled data where each network record is already classified as normal or attack.

Common algorithms include:

- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- Naïve Bayes
- Logistic Regression

Although supervised learning provides high accuracy, it requires large labeled datasets, which may not always be available.

- Unsupervised Learning

Unsupervised learning does not require labeled training data. Instead, it identifies hidden patterns and anomalies directly from the data.

Common algorithms include:

- Isolation Forest
- DBSCAN
- K-Means Clustering
- Local Outlier Factor (LOF)

Our project uses Isolation Forest because it efficiently detects anomalies by isolating unusual observations.

2.4 NSL-KDD Dataset

The NSL-KDD dataset is an improved version of the KDD Cup 1999 dataset. It was developed to overcome redundancy and imbalance problems present in the original dataset.

The dataset contains both normal and attack traffic records with 41 network features. These features describe different aspects of network connections, such as protocol type, service, source bytes, destination bytes, and connection duration.

The NSL-KDD dataset is widely used for evaluating intrusion detection systems because it provides a balanced and standardized benchmark.

2.5 Isolation Forest Algorithm

Isolation Forest is an unsupervised anomaly detection algorithm proposed by Liu, Ting, and Zhou (2008). Unlike distance-based methods, Isolation Forest detects anomalies by randomly partitioning the data.

Since anomalous data points are rare and significantly different from normal observations, they require fewer partitions to become isolated.

The algorithm offers several advantages:

- Fast execution.
- Suitable for high-dimensional datasets.
- Does not require labeled training data.
- Efficient for large datasets.

Because of these characteristics, Isolation Forest is widely used in fraud detection, cybersecurity, medical diagnosis, and network intrusion detection.

2.6 Research Gap

Most traditional intrusion detection systems depend on predefined attack signatures and fail to identify new or unknown cyber threats. Many supervised machine learning methods require a large amount of labelled training data, which is expensive and time-consuming to prepare.

There is a need for an efficient anomaly detection system capable of identifying suspicious network behaviour without relying entirely on labelled datasets.

Unsupervised learning techniques such as Isolation Forest provide a promising solution by detecting anomalies based on deviations from normal traffic patterns.

This project addresses this research gap by implementing an AI-based Network Intrusion Detection System using the Isolation Forest algorithm on the NSL-KDD dataset.

2.7 Summary

This chapter reviewed existing intrusion detection techniques and machine learning approaches. Traditional signature-based systems perform well for known attacks but struggle to identify unknown threats. Machine learning-based anomaly detection offers a more flexible and intelligent solution.

Among the available unsupervised learning algorithms, Isolation Forest was selected for this project due to its efficiency, scalability, and ability to detect anomalous network activities without requiring labelled training data.

The next chapter presents the problem statement, objectives, and scope of the proposed system.

CHAPTER 3

PROBLEM STATEMENT, OBJECTIVES AND SYSTEM REQUIREMENTS

3.1 Introduction

With the increasing use of computer networks in education, banking, healthcare, industries, and cloud computing, protecting network resources has become a major challenge. Every day, organizations exchange huge volumes of sensitive information through networks. This makes them attractive targets for cybercriminals who attempt to steal confidential data, disrupt services, or gain unauthorized access.

Traditional Intrusion Detection Systems (IDS) mainly depend on predefined attack signatures to identify malicious activities. Although these systems perform well for known attacks, they often fail to detect new or previously unseen threats. As cyberattacks become more advanced, there is a growing need for intelligent systems capable of identifying abnormal network behavior automatically.

The proposed project addresses this problem by developing an AI-based Network Intrusion Detection System using the Isolation Forest algorithm, an unsupervised machine learning technique that detects anomalies without relying on labeled training data.

3.2 Problem Statement

Modern computer networks generate a large volume of traffic every second, making manual monitoring practically impossible. Existing signature-based intrusion detection systems can effectively identify known attacks but fail to recognize new or zero-day attacks. Additionally, preparing labeled datasets for supervised machine learning is expensive and time-consuming.

Therefore, there is a need for an intelligent intrusion detection system capable of automatically identifying suspicious network activities by learning the normal behavior of network traffic. Such a system should improve detection efficiency while reducing dependence on predefined attack signatures.

3.3 Objectives

The primary objective of this project is to develop an AI-based Network Intrusion Detection System using the Isolation Forest algorithm to detect anomalous network traffic.

The specific objectives are:

- To study the fundamentals of intrusion detection systems.
- To understand the NSL-KDD dataset and its features.
- To preprocess the dataset using data cleaning, label encoding, and feature scaling techniques.
- To train an Isolation Forest model for anomaly detection.
- To identify suspicious network activities without using labeled training data during model training.
- To evaluate the model using a confusion matrix and classification report.
- To visualize the prediction results using graphs such as pie charts, bar charts, and histograms.
- To save the trained model for future use.

3.4 Scope of the Project

The scope of this project is focused on detecting anomalous network activities using unsupervised machine learning techniques.

The developed system can be applied in:

- Educational institutions for learning cybersecurity concepts.
- Research in anomaly detection and machine learning.
- Enterprise network monitoring.
- Banking and financial networks.
- Government organizations.
- Cloud computing environments.
- Small and medium-scale computer networks.

The current implementation uses the NSL-KDD dataset in an offline environment. In the future, the same methodology can be extended to analyze real-time network traffic.

3.5 Proposed Solution

The proposed solution uses machine learning to identify suspicious network behavior.

The process begins with loading the NSL-KDD dataset into Python. The dataset is preprocessed by checking missing values, removing duplicates, encoding categorical attributes, and standardizing numerical features.

The processed data is then provided to the Isolation Forest algorithm. The algorithm learns the characteristics of normal network traffic and isolates unusual observations as anomalies.

The generated predictions are evaluated using performance metrics and visualized through different graphs to understand the effectiveness of the proposed intrusion detection system.

3.6 Functional Requirements

The system should perform the following functions:

- Load the NSL-KDD dataset.
- Display dataset information.
- Perform data preprocessing.
- Encode categorical variables.
- Scale numerical features.
- Train the Isolation Forest model.
- Detect anomalies.
- Generate predictions.
- Evaluate model performance.
- Display graphical visualizations.
- Save the trained machine learning model.

3.7 Non-Functional Requirement

The proposed system should satisfy the following non-functional requirements:

- Performance
- The system should detect anomalies efficiently with minimum processing time.
- Reliability
- The prediction results should remain consistent for the same input dataset.
- Scalability
- The system should support larger datasets with minimal modification.
- Usability
- The interface should be simple enough for students and researchers to understand.
- Maintainability

The code should be modular and well-documented to support future improvements.

3.8 Hardware Requirements

The project can be executed using a standard personal computer.

Minimum Hardware Requirements:

- Processor: Intel Core i3 or higher
- RAM: 4 GB (8 GB recommended)
- Storage: 10 GB free space
- Operating System: Windows 10/11, Linux, or macOS

3.9 Software Requirements

The software required for implementing the project includes:

Software	Purpose
Python 3.x	Programming language
Jupyter Notebook	Development environment
Pandas	Data manipulation
NumPy	Numerical computations
Matplotlib	Data visualization
Scikit-learn	Machine learning algorithms
Joblib	Saving trained models

3.10 Advantages of the Proposed System

The proposed system offers several advantages:

- Detects unknown attacks using anomaly detection.
- Does not require labeled training data.
- Fast execution using the Isolation Forest algorithm.
- Easy to implement and understand.
- Provides visual analysis through graphs.
- Suitable for educational and research purposes.
- Can be extended for real-time intrusion detection.

3.11 Limitations

Although the proposed system performs effectively, it has certain limitations:

- The model is trained using an offline dataset rather than live network traffic.
- Performance depends on the quality of the dataset.
- The contamination parameter must be chosen carefully.
- Only one machine learning algorithm (Isolation Forest) is implemented in the current version.

3.12 Chapter Summary

This chapter presented the problem statement, objectives, scope, system requirements, and the proposed solution for the AI-based Network Intrusion Detection System. It also discussed the advantages and limitations of the proposed approach. The next chapter describes the NSL-KDD dataset in detail, including its features, attack categories, preprocessing techniques, and the role of each attribute in intrusion detection.

CHAPTER 4

DATASET DESCRIPTION

4.1 Introduction

The performance of any machine learning model largely depends on the quality of the dataset used for training and evaluation. Since the objective of this project is to detect network intrusions using unsupervised machine learning, selecting a reliable and widely accepted dataset is essential.

This project uses the NSL-KDD dataset, which is one of the most popular benchmark datasets for intrusion detection research. It was developed as an improved version of the KDD Cup 1999 dataset to overcome issues such as duplicate records and class imbalance. The NSL-KDD dataset provides realistic network traffic data that can be used to evaluate intrusion detection systems.

4.2 NSL-KDD Dataset

The NSL-KDD dataset was introduced by Tavallae et al. (2009) as an enhanced version of the KDD Cup 1999 dataset.

It removes redundant records from the original dataset, making machine learning algorithms more effective and producing more reliable evaluation results.

The dataset consists of network connection records collected from simulated network traffic. Each record represents one network connection and contains several features describing the characteristics of that connection.

The dataset includes both normal network traffic and different categories of network attacks.

4.3 Why NSL-KDD was Selected

The NSL-KDD dataset was selected for the following reasons:

- It is widely accepted by researchers in cybersecurity.
- It removes duplicate records present in the KDD Cup 1999 dataset.
- It provides balanced training and testing datasets.
- It contains multiple categories of cyber attacks.
- It is suitable for evaluating anomaly detection algorithms.
- It is freely available for research and educational purposes.

4.4 Dataset Structure

The NSL-KDD dataset contains:

- Approximately 125,973 training records
- 43 columns (including label and difficulty level)
- Each row represents a single network connection.

The features describe different properties of the connection such as:

- Duration
- Protocol used
- Service requested
- Number of bytes transferred
- Error rate
- Number of failed login attempts
- Connection statistics

4.5 Important Features Used

Some of the important attributes in the dataset are explained below.

- Duration

Represents the duration of the network connection in seconds.

- Protocol Type

Specifies the communication protocol used.

Examples: TCP, UDP, ICMP

- Service

Represents the network service requested.

Examples include: HTTP, FTP, SMTP, Telnet, DNS

- Flag

Represents the status of the network connection.

Examples: SF, REJ, S0

- Source Bytes (src_bytes)

Represents the amount of data sent from the source computer to the destination.

- Destination Bytes (dst_bytes)

Represents the amount of data received by the destination computer.

- Count

Represents the number of connections made to the same destination host during a specified time interval.

- Logged In

Indicates whether the login attempt was successful.

1 → Successful login

0 → Failed login

- Label

Represents whether the connection is normal or belongs to a particular attack category.

Examples: normal, Neptune, smurf, satan, back

Although this column exists in the dataset, it is not used for training in this project because the model is based on unsupervised learning. It is used only to evaluate the model's predictions.

4.6 Attack Categories

The NSL-KDD dataset contains several types of cyber attacks, grouped into four major categories.

1. Denial of Service (DoS)

These attacks attempt to make a network service unavailable by overwhelming it with excessive traffic.

Examples: Neptune, Smurf, Back, Teardrop

2. Probe Attacks

Probe attacks attempt to collect information about a network before launching an attack.

Examples: Satan, Ipsweep, Nmap, Portsweep

3. User-to-Root (U2R)

The attacker initially gains normal user access and then attempts to obtain administrator privileges.

Examples: Buffer Overflow, Rootkit, Perl

4. Remote-to-Local (R2L)

The attacker gains unauthorized access to a computer from a remote location.

Examples: Guess Password, FTP Write, Multihop

4.7 Dataset Preprocessing

Before training the machine learning model, the dataset undergoes several preprocessing steps.

- Data Loading

The dataset is loaded into Python using the Pandas library.

- Missing Value Check

The dataset is examined for missing values.

No missing values were found.

- Duplicate Removal

Duplicate records are checked and removed to improve model performance.

- Label Encoding

Categorical features such as:

- Protocol Type
- Service
- Flag

are converted into numerical values because machine learning algorithms cannot process text directly.

- Feature Scaling

The numerical features are standardized using StandardScaler.

Feature scaling ensures that all features have similar ranges, improving the performance of the Isolation Forest algorithm.

4.8 Target Variable

A target variable is created for evaluation purposes.

Normal traffic $\rightarrow 0$

Attack traffic $\rightarrow 1$

Although the project uses unsupervised learning, this target variable is used only for comparing the predicted results with the actual labels.

4.9 Dataset Visualization

Several visualizations are generated to better understand the dataset.

These include:

- Actual Label Distribution
- Protocol Type Distribution
- Source Bytes Distribution
- Destination Bytes Distribution
- Prediction Distribution
- Confusion Matrix

These visualizations help analyze the characteristics of network traffic and evaluate the model's performance.

4.10 Advantages of the Dataset

The NSL-KDD dataset provides several advantages:

- Standard benchmark dataset
- Balanced data distribution.
- Contains multiple attack categories.
- Widely used in research.
- Suitable for supervised and unsupervised learning.
- Easy to preprocess and analyze.

4.11 Limitations of the Dataset

Despite its popularity, the dataset has some limitations:

- It is based on simulated network traffic rather than modern real-world traffic.
- Some attack patterns are outdated.
- It may not fully represent current cybersecurity threats.
- Performance on the dataset may differ from real-time environments.

4.12 Chapter Summary

This chapter described the NSL-KDD dataset used in the project. It discussed the dataset structure, important features, attack categories, preprocessing techniques, advantages, and limitations. Proper preprocessing, including label encoding and feature scaling, prepares the data for training the Isolation Forest model. The next chapter explains the proposed methodology, detailing each step of the implementation from data loading to anomaly detection and result visualization.

CHAPTER 5

PROPOSED METHODOLOGY

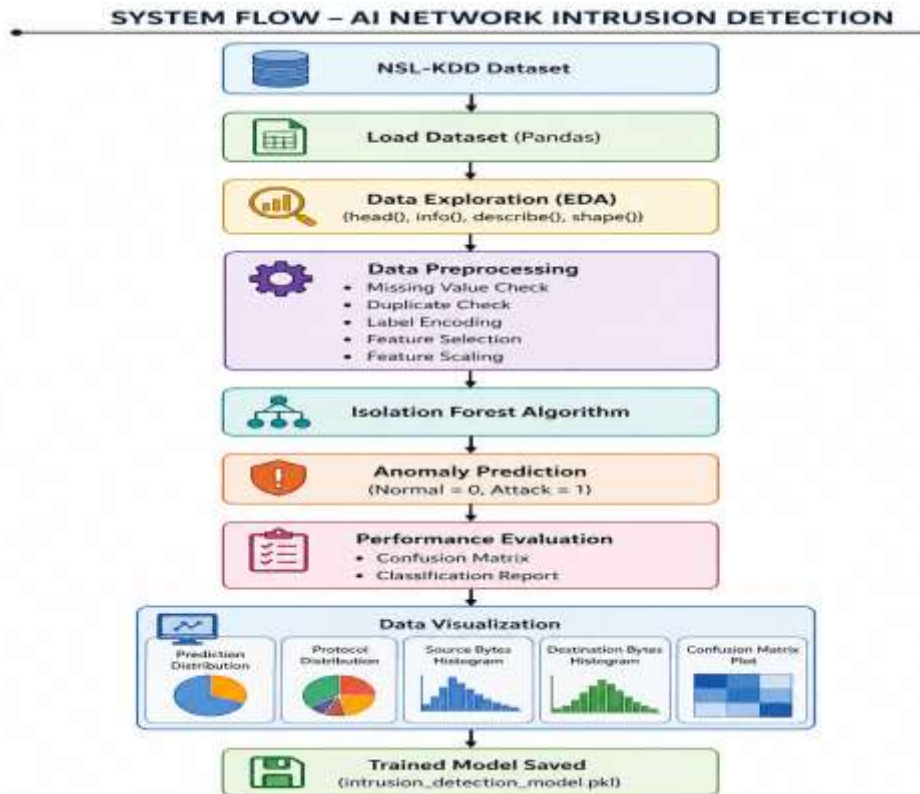
5.1 Introduction

The proposed methodology describes the sequence of steps followed to develop the AI-Based Network Intrusion Detection System. The system uses the NSL-KDD dataset and the Isolation Forest algorithm to identify anomalous network activities. Before training the machine learning model, the dataset is preprocessed to improve data quality and ensure compatibility with the algorithm.

The complete methodology consists of data collection, preprocessing, feature engineering, model training, anomaly prediction, performance evaluation, and visualization of results.

5.2 System Workflow

The overall workflow of the proposed system is shown below.



5.3 Data Collection

The first step of the methodology is collecting the dataset.

The NSL-KDD dataset was selected because it is a standard benchmark dataset for intrusion detection research. It contains network traffic records representing both normal and malicious network activities.

Each record includes multiple network features describing the characteristics of a network connection. The dataset is loaded into Python using the Pandas library.

Example:

```
df = pd.read_csv("KDDTrain+.txt", names=column_names)
```

5.4 Data Exploration

After loading the dataset, it is explored to understand its structure.

The following operations are performed:

- Display first five records
- Display dataset dimensions
- View column names
- Check data types
- Generate statistical summary

Functions used:

- `df.head()`
- `df.shape`
- `df.info()`
- `df.describe()`

These operations help understand the dataset before preprocessing.

5.5 Data Preprocessing

Machine learning algorithms require clean and properly formatted data. Therefore, preprocessing is performed before training the model.

The preprocessing includes:

- Missing Value Detection

The dataset is checked for missing values.

```
df.isnull().sum()
```

No missing values were observed.

- Duplicate Record Detection

Duplicate records are checked using:

```
df.duplicated().sum()
```

Duplicate records are removed whenever necessary to improve model performance.

5.6 Target Variable Creation

Although Isolation Forest is an unsupervised algorithm, the dataset labels are converted into binary values for evaluation purposes.

The conversion is performed as:

Normal $\rightarrow$ 0

Attack $\rightarrow$ 1

Example:

```
df['target'] = df['label'].apply(lambda x: 0 if x=='normal' else 1)
```

This target variable is used only to compare predictions with the actual dataset labels.

5.7 Label Encoding

Several columns contain categorical values such as:

- protocol_type
- service
- flag

Machine learning algorithms cannot process textual data directly.

Therefore, Label Encoder converts text into numerical values.

Example:

- Before Encoding
- protocol_type
- tcp
- udp

- icmp

Python code:

- encoder = LabelEncoder()
- df['protocol_type'] = encoder.fit_transform(df['protocol_type'])
- df['service'] = encoder.fit_transform(df['service'])
- df['flag'] = encoder.fit_transform(df['flag'])

5.8 Feature Selection

The label and difficulty columns are removed because they are not used for training.

```
X = df.drop(['label','difficulty','target'], axis=1)
```

The remaining features become the input for the machine learning model.

5.9 Feature Scaling

Different features have different numerical ranges.

Example:

Feature	Value
Duration	5
Source Bytes	45000

Such differences may affect the learning process.

Therefore, StandardScaler is applied.

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Feature scaling standardizes all features so that they contribute equally during model training.

5.10 Isolation Forest Algorithm

Isolation Forest is an unsupervised anomaly detection algorithm.

Instead of learning attack patterns, it isolates unusual observations.

Since anomalies are rare, they require fewer partitions to become isolated.

The algorithm constructs multiple random trees and calculates anomaly scores.

Observations that are isolated quickly are classified as anomalies.

Python implementation:

```
model = IsolationForest(  
    contamination=0.2,  
    random_state=42)  
model.fit(X_scaled  
)
```

The contamination parameter indicates the expected proportion of anomalies in the dataset.

5.11 Prediction

After training, the model predicts whether each network connection is normal or anomalous.

Prediction:

```
predictions = model.predict(X_scaled)
```

Isolation Forest returns:

Output	Meaning
1	Normal
-1	Anomaly

The predictions are converted into binary values:

```
predictions = np.where(predictions==1,0,1)
```

Result:

Normal $\rightarrow$ 0

Attack $\rightarrow$ 1

5.12 Performance Evaluation

The predicted results are compared with the actual labels.

The following metrics are calculated: Confusion Matrix

The confusion matrix shows:

- True Positives
- True Negatives
- False Positives
- False Negatives

Python:

```
confusion_matrix(y,predictions)
```

Classification Report

The classification report calculates:

- Precision
- Recall
- F1 Score
- Accuracy

Python:

```
classification_report(y,predictions)
```

These metrics evaluate the effectiveness of the intrusion detection system.

5.13 Data Visualization

Several graphs are generated to visualize the model results.

The visualizations include:

- Prediction Distribution

Shows the percentage of normal and anomalous network traffic.

- Prediction Histogram

Displays the frequency of predicted normal and attack records.

- Confusion Matrix

Visualizes prediction accuracy.

- Actual Label Distribution

Displays the distribution of original dataset labels.

- Protocol Distribution

Shows the number of TCP, UDP, and ICMP connections.

- Source Bytes Distribution

Displays the distribution of bytes sent from the source.

- Destination Bytes Distribution

Shows the distribution of bytes received by the destination.

These graphs improve understanding of the model's performance and the characteristics of the dataset.

5.14 Model Saving

After successful training, the model is stored using the Joblib library.

```
joblib.dump(model,"intrusion_detection_model.pkl")
```


Saving the trained model avoids retraining whenever predictions are needed in the future.

5.15 Advantages of the Proposed Methodology

The proposed methodology offers several advantages:

- Detects unknown attacks using anomaly detection.
- Does not require labeled data for training.
- Efficient for large datasets.
- Simple and easy to implement.
- Provides multiple visualizations for analysis.
- Can be extended to real-time intrusion detection systems.

5.16 Chapter Summary

This chapter described the complete methodology adopted for developing the AI-Based Network Intrusion Detection System. It explained each implementation step, including data collection, preprocessing, feature scaling, Isolation Forest training, prediction, evaluation, visualization, and model saving. These steps collectively enable the system to detect anomalous network activities effectively.

CHAPTER 6

IMPLEMENTATION

6.1 Introduction

This chapter explains the implementation of the proposed AI-Based Network Intrusion Detection System. The system was developed using Python in Jupyter Notebook. Various Python libraries such as Pandas, NumPy, Matplotlib, Scikit-learn, and Joblib were used for data preprocessing, model development, evaluation, and visualization.

The implementation follows a systematic approach consisting of dataset loading, preprocessing, feature engineering, model training, prediction, performance evaluation, visualization, and model saving.

6.2 Software Environment

The project was implemented using the following software tools.

Software	Purpose
Python 3.x	Programming language
Jupyter Notebook	Development environment
Pandas	Data manipulation
NumPy	Numerical computations
Matplotlib	 Data visualization
Scikit-learn	Machine learning algorithms
<u>Joblib</u>	Saving trained models

6.3 Importing Required Libraries

The first step in the implementation is importing all the required Python libraries.

The following libraries were used:

- Pandas
- NumPy
- Matplotlib
- LabelEncoder
- StandardScaler
- IsolationForest
- Confusion Matrix
- Classification Report
- Joblib

The import statements prepare the environment for data analysis and machine learning implementation.

6.4 Loading the Dataset

The NSL-KDD dataset was loaded into Jupyter Notebook using the Pandas `read_csv()` function.

The dataset contains network traffic records and their corresponding features.

After loading, the first five records were displayed using `head()` to verify that the dataset had been imported successfully.

```
In [7]: df.head()
```

```
Out[7]:
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_same_srv_rate	dst_host_diff_srv_rate
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	0.17	0.03
1	0	udp	other	SF	146	0	0	0	0	0	...	0.00	0.60
2	0	tcp	private	S0	0	0	0	0	0	0	...	0.10	0.05
3	0	tcp	http	SF	232	8153	0	0	0	0	...	1.00	0.00
4	0	tcp	http	SF	199	420	0	0	0	0	...	1.00	0.00

6.5 Exploring the Dataset

Before preprocessing, the dataset was explored to understand its characteristics.

The following functions were executed:

- `head()`
- `shape`
- `columns`
- `info()`
- `describe()`

These functions provided information about:

- Number of records
- Number of features
- Data types

- Statistical summary

This step helped in understanding the dataset before applying machine learning algorithms.

6.6 Checking Missing Values

The dataset was checked for missing values using: `df.isnull().sum()`

The output confirmed that no missing values were present in the dataset. This eliminated the need for missing value imputation.

```
In [11]: df.isnull().sum()

Out[11]: duration      0
protocol_type      0
service            0
flag               0
src_bytes          0
dst_bytes          0
land               0
wrong_fragment     0
urgent             0
hot                0
num_failed_logins  0
logged_in          0
num_compromised    0
root_shell         0
su_attempted       0
num_root           0
num_file_creations 0
num_shells         0
num_access_files   0
num_outbound_cmds  0
is_host_login      0
is_guest_login     0
...
```

6.7 Checking Duplicate Records

Duplicate records were checked to avoid redundant information.

The duplicate check ensured that repeated observations did not affect model performance.

Since duplicate records were minimal, they were removed where necessary.

```
In [13]: df.duplicated().sum()
```

```
Out[13]: np.int64(0)
```

6.8 Label Distribution

The distribution of attack labels was examined using the `value_counts()` function.

This analysis showed the frequency of normal and different attack categories present in the dataset.

The information was useful for understanding the characteristics of the dataset.

```
In [14]: df['label'].value_counts()
```

```
Out[14]: label
normal          67343
neptune         41214
satan           3633
ipsweep         3599
portsweep       2931
smurf           2646
nmap            1493
back            956
teardrop        892
warezclient     890
pod             201
guess_passwd    53
buffer_overflow 30
warezmaster     20
1 -- 1         10
```

6.9 Target Variable Creation

Since Isolation Forest is an unsupervised learning algorithm, the original attack labels were converted into binary values only for evaluation purposes.

The conversion was performed as:

Normal $\rightarrow$ 0

Attack $\rightarrow$ 1

The target variable was stored in a new column named target.

This allowed comparison between actual labels and predicted results after model training.

6.10 Label Encoding

The dataset contained categorical features such as:

- Protocol Type
- Service
- Flag

Machine learning algorithms require numerical input.

Therefore, Label Encoding was applied to convert textual values into integer values.

Example:

Before Encoding	After Encoding
Protocol	Protocol
Tcp	0
Udp	1
Icmp	2

This transformation enabled the Isolation Forest algorithm to process the data.

6.11 Feature Scaling

The dataset contained features with different numerical ranges.

For example:

Feature	Range
Duration	0–500
Source Bytes	0–1,000,000

Large differences in feature values may negatively affect machine learning algorithms.

To overcome this issue, StandardScaler was applied.

Standardization transformed each feature to have a mean of zero and a standard deviation of one.

This improved the performance of the Isolation Forest model.

6.12 Model Training

The preprocessed dataset was provided to the Isolation Forest algorithm.

The model was initialized with:

Contamination = 0.2

Random State = 42

The contamination parameter specifies the expected percentage of anomalous observations in the dataset.

The model learned the characteristics of normal network traffic without using attack labels.

6.13 Prediction

After training, the model predicted whether each network connection was normal or anomalous.

The model returned:

- 1 for Normal Traffic
- -1 for Anomalous Traffic

The predictions were converted into binary values:

Normal $\rightarrow$ 0

Attack $\rightarrow$ 1

These predictions were stored for evaluation.

6.14 Model Evaluation

The performance of the model was evaluated using two methods.

- Confusion Matrix

The confusion matrix compared actual labels with predicted labels.

It provided information about:

- True Positives
- True Negatives
- False Positives
- False Negatives

```
print(confusion_matrix(y, predictions))
```

```
[[59470  7873]
 [41308 17322]]
```

The confusion matrix helped evaluate the correctness of the predictions.

- Classification Report

The classification report calculated:

- Precision
- Recall
- F1-score
- Support

```
: print(classification_report(y, predictions))
```

	precision	recall	f1-score	support
0	0.59	0.88	0.71	67343
1	0.69	0.30	0.41	58630
accuracy			0.61	125973
macro avg	0.64	0.59	0.56	125973
weighted avg	0.64	0.61	0.57	125973

These metrics measured the performance of the intrusion detection model.

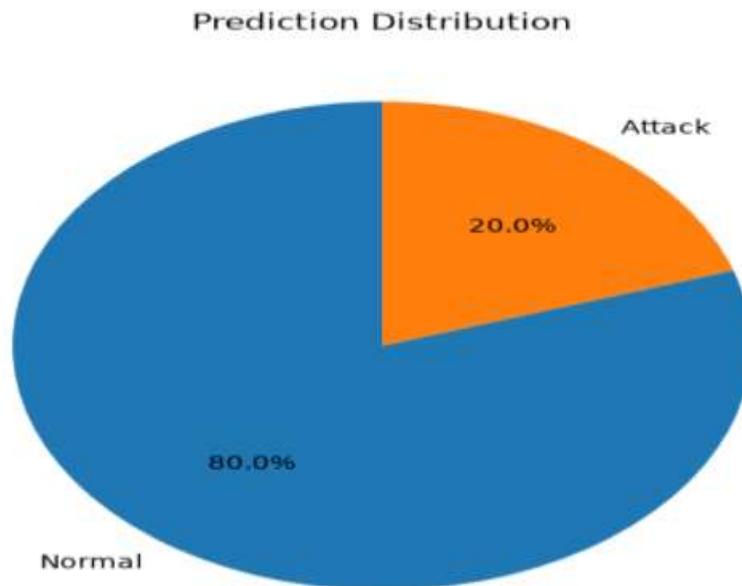
6.15 Visualization

Several graphs were generated to understand the dataset and prediction results.

The generated graphs include:

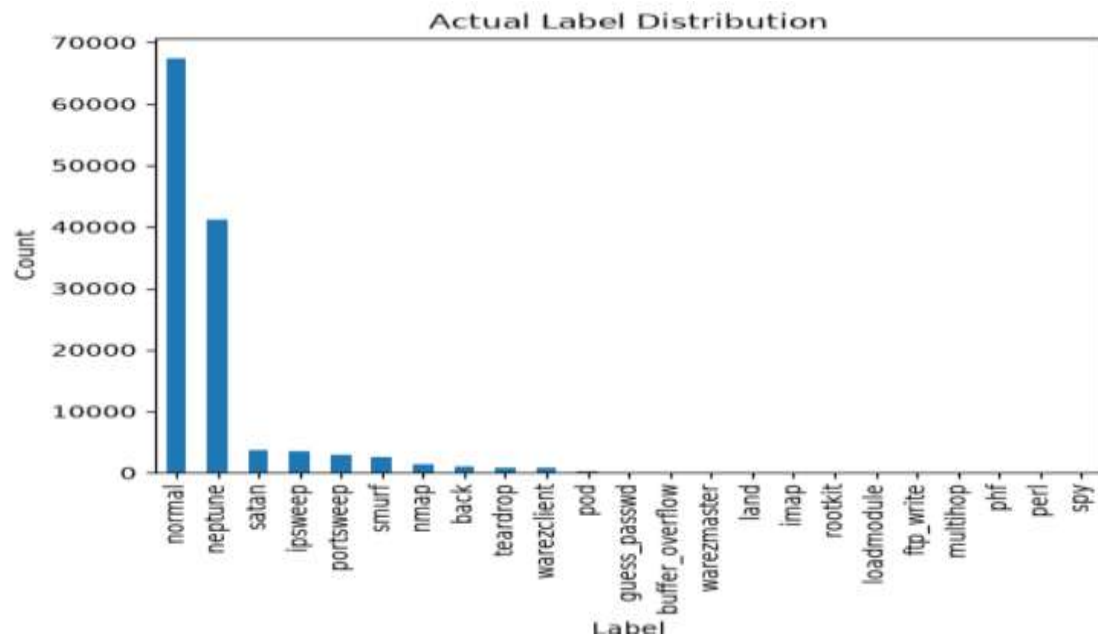
- Prediction Distribution Pie Chart

Shows the percentage of predicted normal and attack traffic.



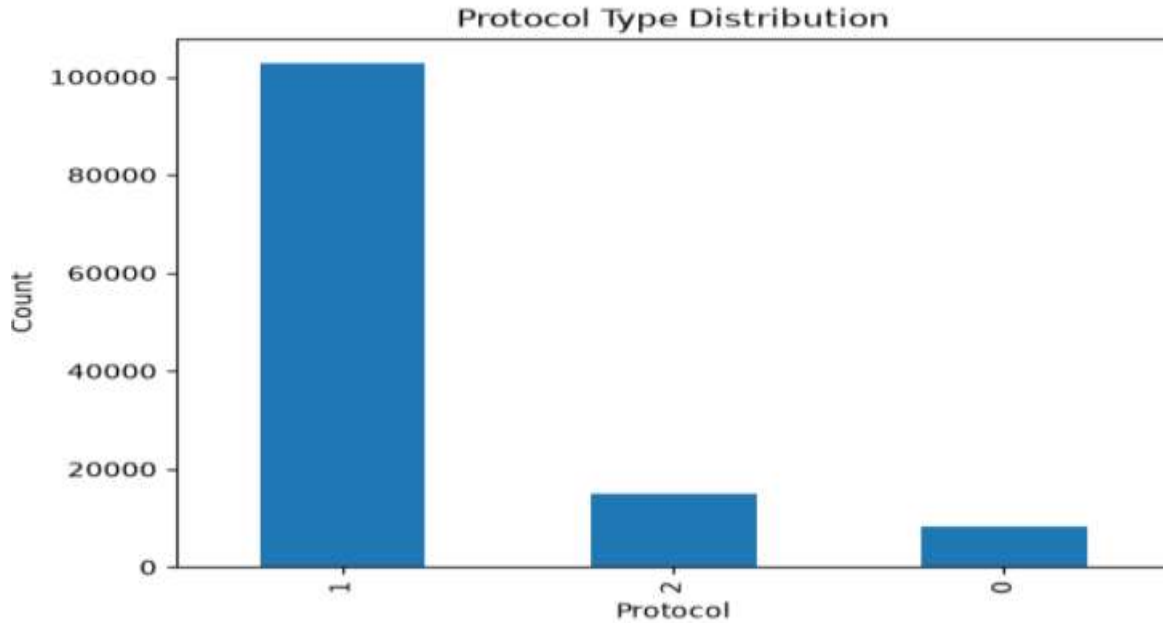
- Actual Label Distribution

Displays the distribution of different attack categories present in the dataset.



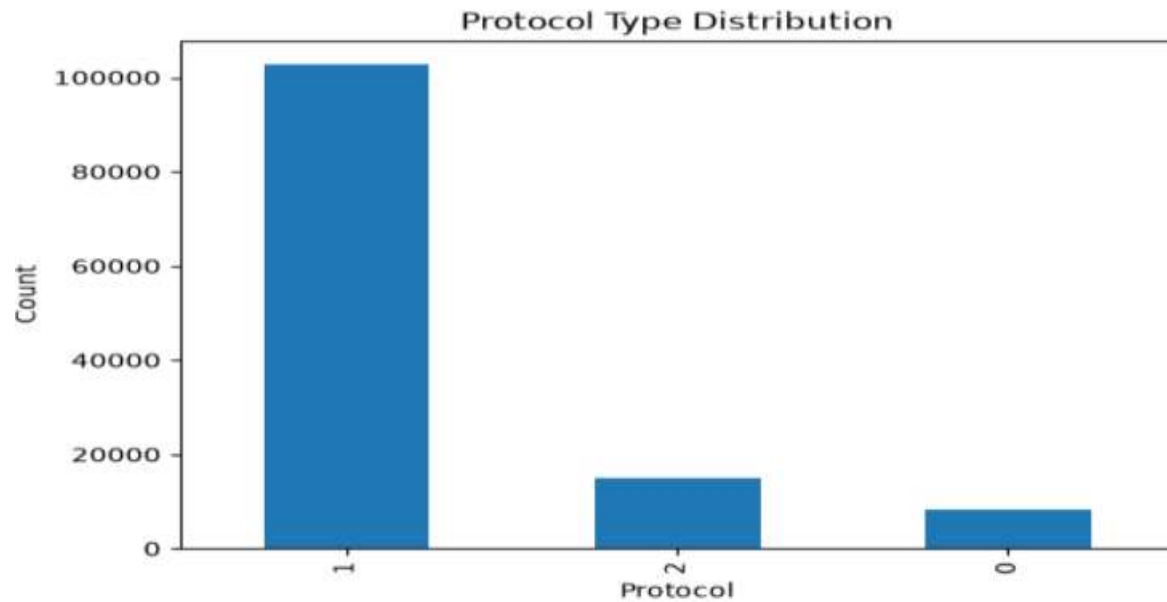
- Protocol Distribution

Shows the number of TCP, UDP, and ICMP connections.



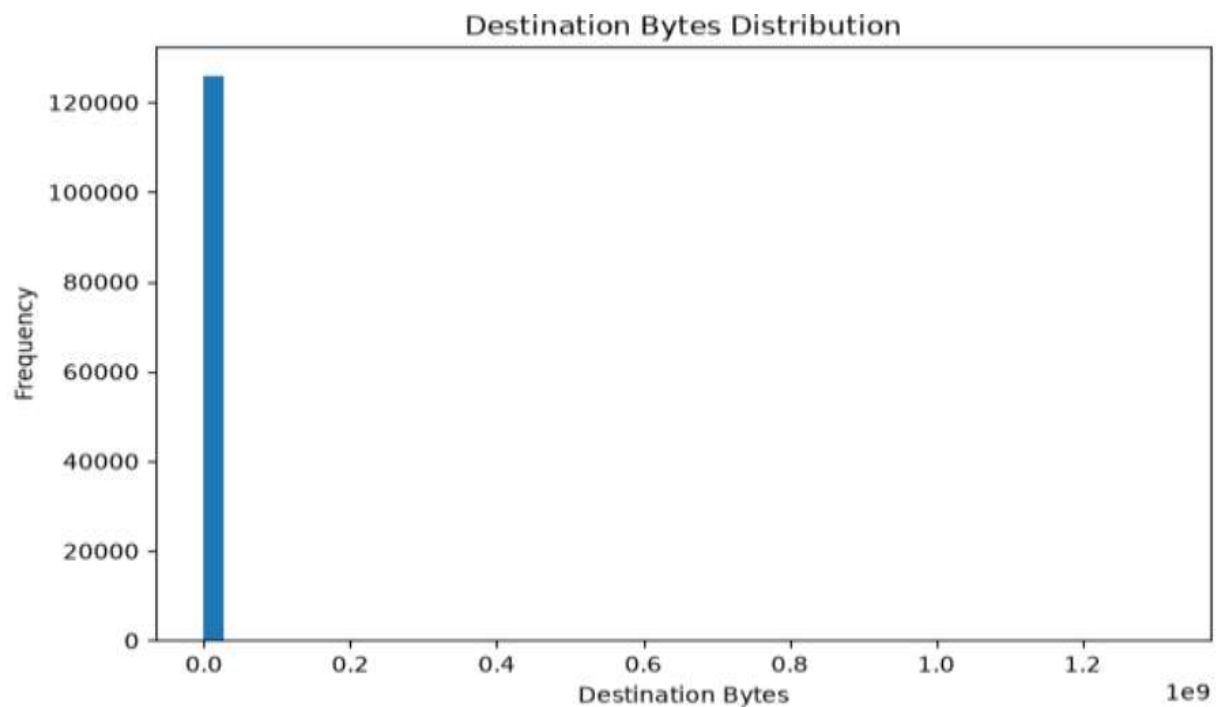
- Source Bytes Distribution

Displays the distribution of source bytes.



- Destination Bytes Distribution

Displays the distribution of destination bytes.



6.16 Saving the Model

The trained model was saved using the Joblib library.

Saving the model allows it to be reused without retraining.

The model was stored as:

`intrusion_detection_model.pkl`

This file can later be loaded for prediction in future applications.

6.17 Advantages of the Implementation

The implemented system offers several benefits:

- Detects abnormal network activities automatically.
- Does not require labeled training data.
- Provides graphical representation of results.
- Easy to understand and implement.
- Uses a standard benchmark dataset.
- Can be extended for real-time intrusion detection.

6.18 Chapter Summary

This chapter described the complete implementation of the proposed AI-Based Network Intrusion Detection System. The implementation included importing libraries, loading the dataset, preprocessing, feature engineering, model training, anomaly prediction, evaluation, visualization, and saving the trained model. The implementation successfully demonstrated the effectiveness of the Isolation Forest algorithm in identifying anomalous network traffic.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Introduction

This chapter presents the experimental results obtained from the proposed AI-Based Network Intrusion Detection System. The implementation was carried out using Python and Jupyter Notebook with the NSL-KDD dataset. After preprocessing the dataset, the Isolation Forest algorithm was trained to identify anomalous network traffic.

The model predictions were evaluated using a confusion matrix and classification report. In addition, several graphical visualizations were generated to analyze the characteristics of the dataset and understand the performance of the model.

7.2 Dataset Analysis

Initially, the NSL-KDD dataset was explored to understand its structure and properties.

The following analyses were performed:

- Displaying the first five records
- Checking dataset dimensions
- Viewing column names
- Identifying data types
- Statistical summary
- Missing value analysis
- Duplicate record analysis

The dataset contained 125,973 records and 43 attributes, making it suitable for training and evaluating the anomaly detection model.

7.3 Missing Value Analysis

The dataset was examined for missing values using the `isnull().sum()` function.

- Observation

No missing values were found in any of the dataset attributes.

- Discussion

Since the dataset was complete, no missing value handling techniques such as mean imputation or row deletion were required. This improved the reliability of the training process.

```
In [11]: df.isnull().sum()
```

```
Out[11]: duration          0
protocol_type          0
service               0
flag                 0
src_bytes            0
dst_bytes            0
land                 0
wrong_fragment       0
urgent               0
hot                 0
num_failed_logins    0
logged_in            0
num_compromised      0
root_shell           0
su_attempted         0
num_root             0
num_file_creations   0
num_shells           0
num_access_files     0
num_outbound_cmds    0
is_host_login        0
is_guest_login        0
count               0
srv_count            0
serror_rate          0
```

7.4 Duplicate Record Analysis

Duplicate records were checked before model training.

- Observation

The dataset contained very few duplicate records.

- Discussion

Removing duplicate entries helped reduce redundancy and prevented repeated information from influencing the machine learning model.

```
[13]: df.duplicated().sum()
```

```
Out[13]: np.int64(0)
```

7.5 Actual Label Distribution

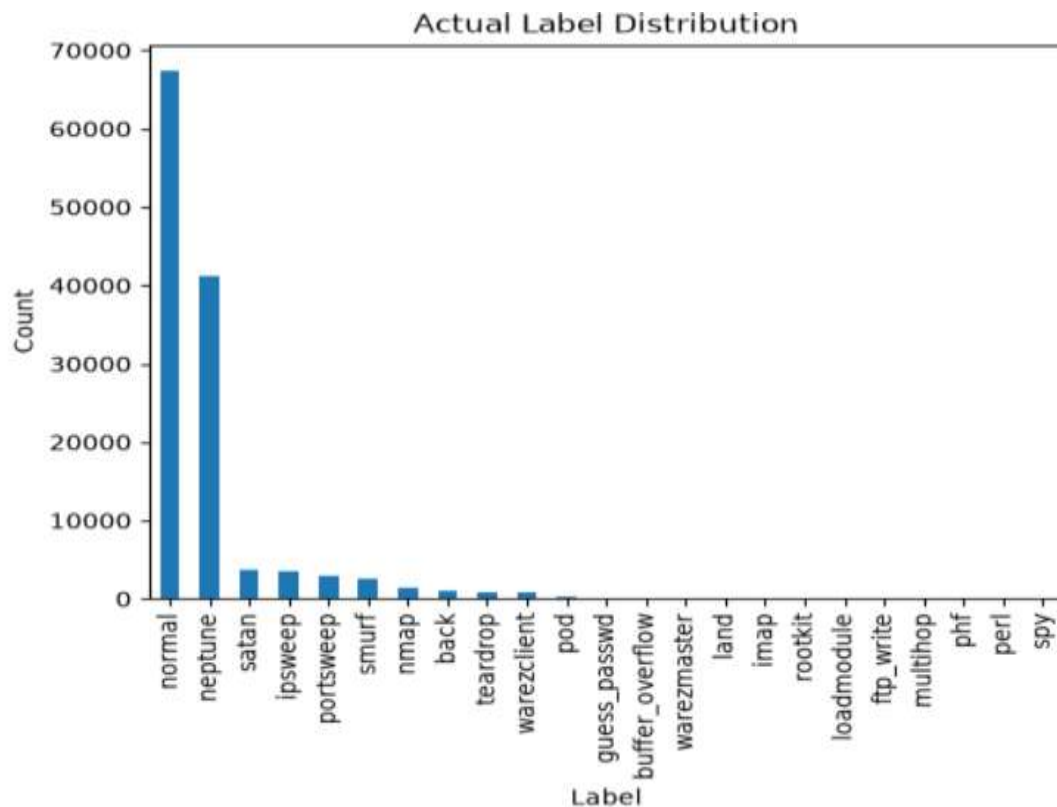
The label distribution graph illustrates the frequency of different attack categories and normal network traffic present in the dataset.

- Observation

The dataset contains normal traffic along with multiple attack types such as DoS, Probe, R2L, and U2R attacks.

- Discussion

Understanding the label distribution provides an overview of the dataset composition. Although the labels were not used during training, they were used later for evaluating the model predictions.



7.6 Protocol Type Distribution

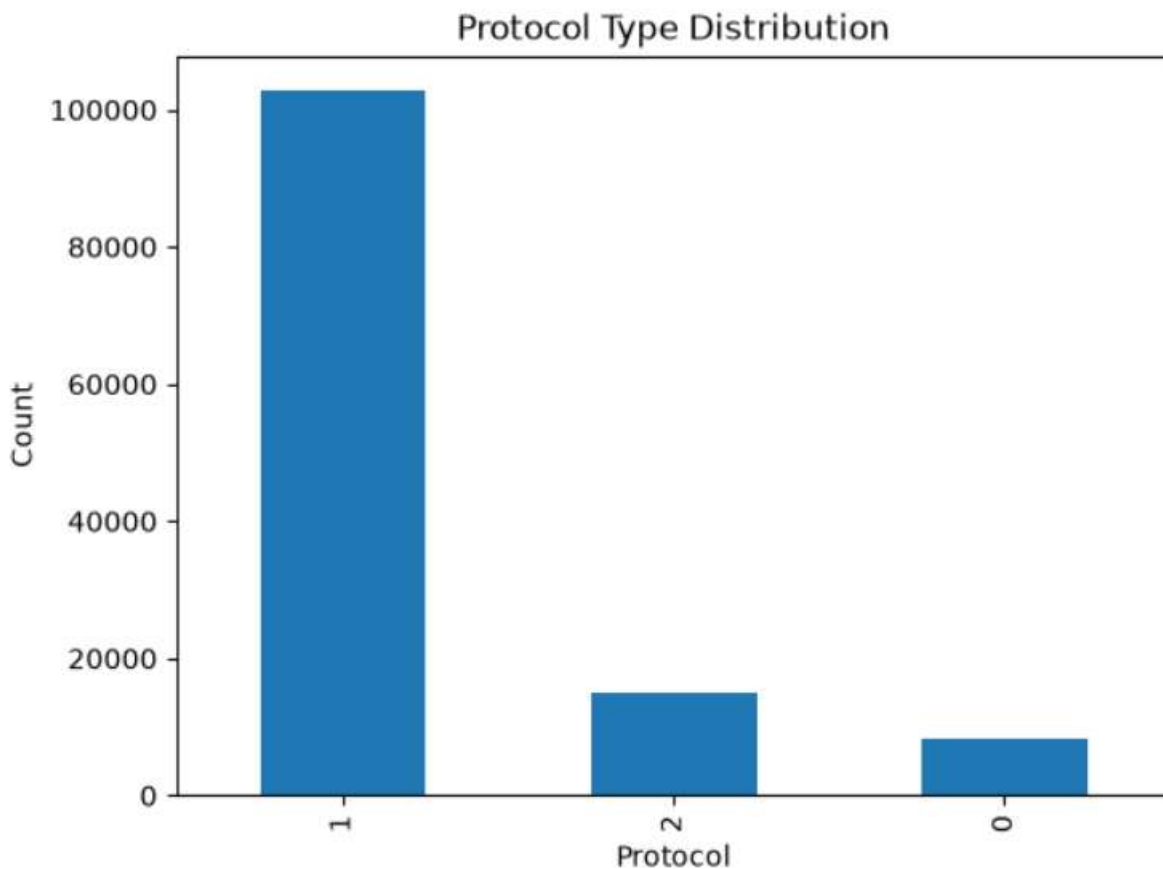
The protocol distribution graph shows the number of network connections using different communication protocols.

The major protocols observed were:

- TCP

- UDP
- ICMP
-
- Discussion

TCP traffic constituted the majority of network connections, while UDP and ICMP represented comparatively fewer connections. This distribution reflects typical network communication patterns.



7.7 Source Bytes Distribution

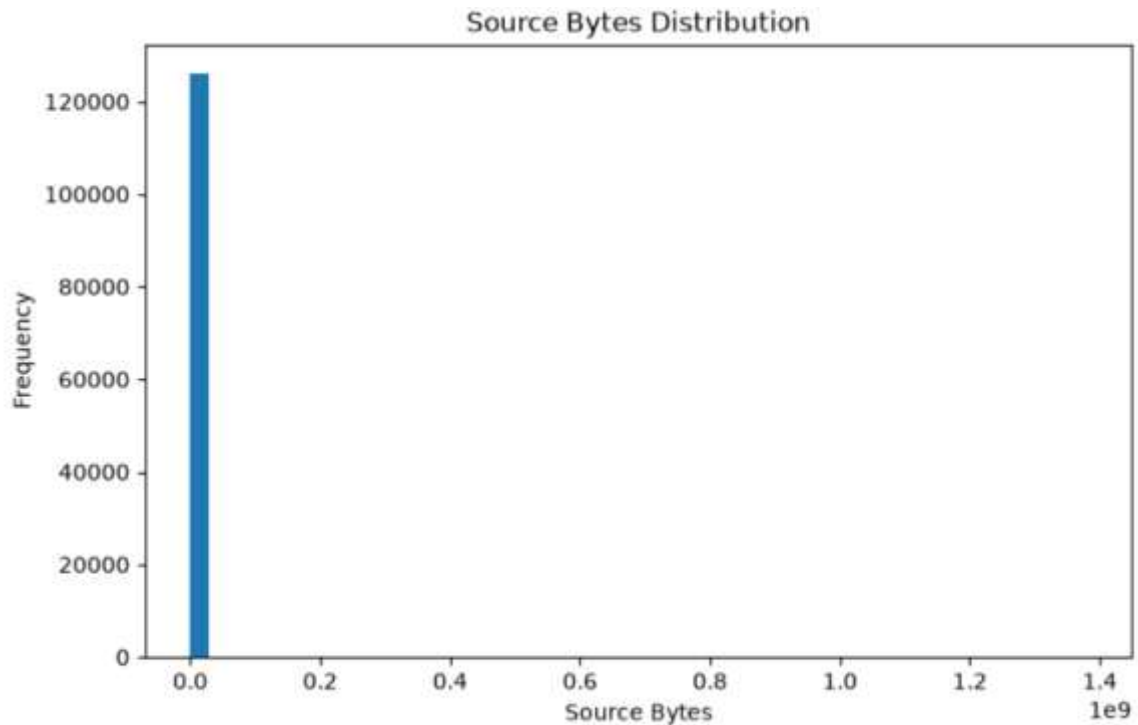
The histogram of source bytes illustrates the amount of data transmitted from the source host.

- Observation

Most network connections transferred relatively small amounts of data, while a small number of connections transmitted very large volumes.

- Discussion

This indicates that network traffic is highly skewed, which is common in real-world datasets. Extremely large values may correspond to abnormal or suspicious network activities.



7.8 Destination Bytes Distribution

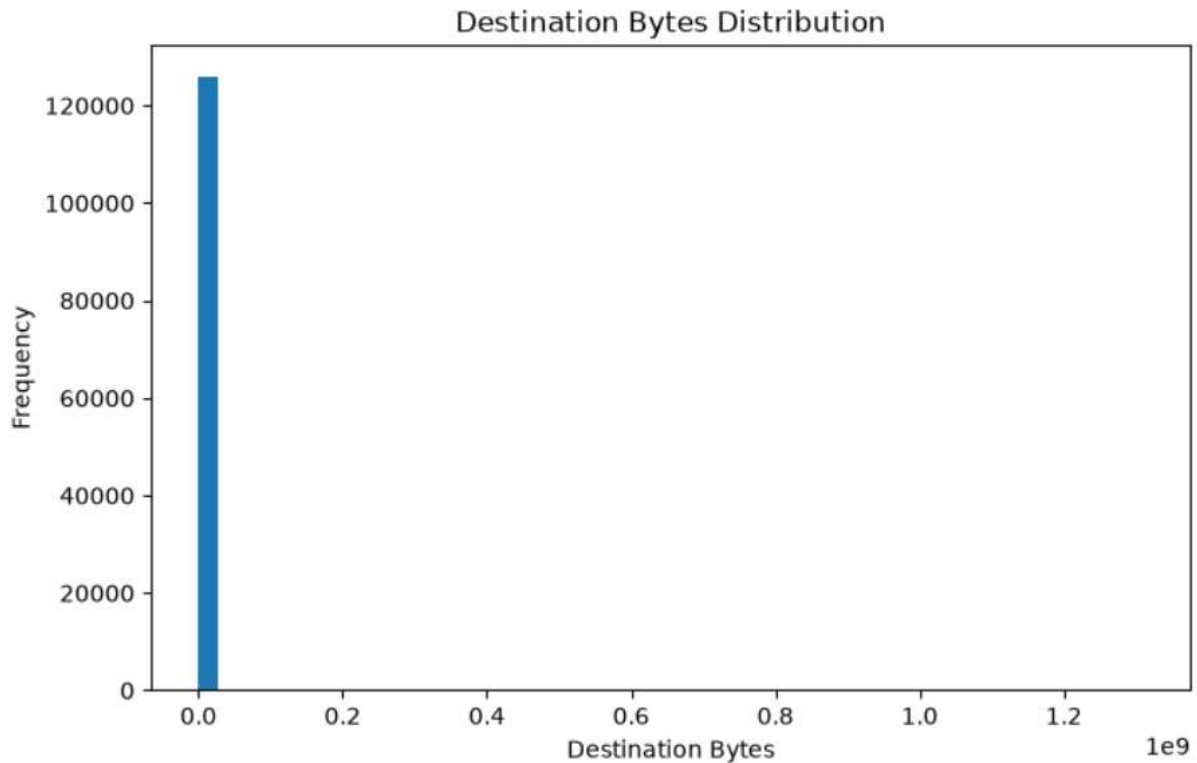
The histogram of destination bytes represents the amount of data received by the destination host.

- Observation

Most observations were concentrated at lower byte values, with a few extremely large values.

- Discussion

The presence of outliers suggests variations in network communication patterns and highlights the importance of anomaly detection algorithms.



7.9 Prediction Distribution

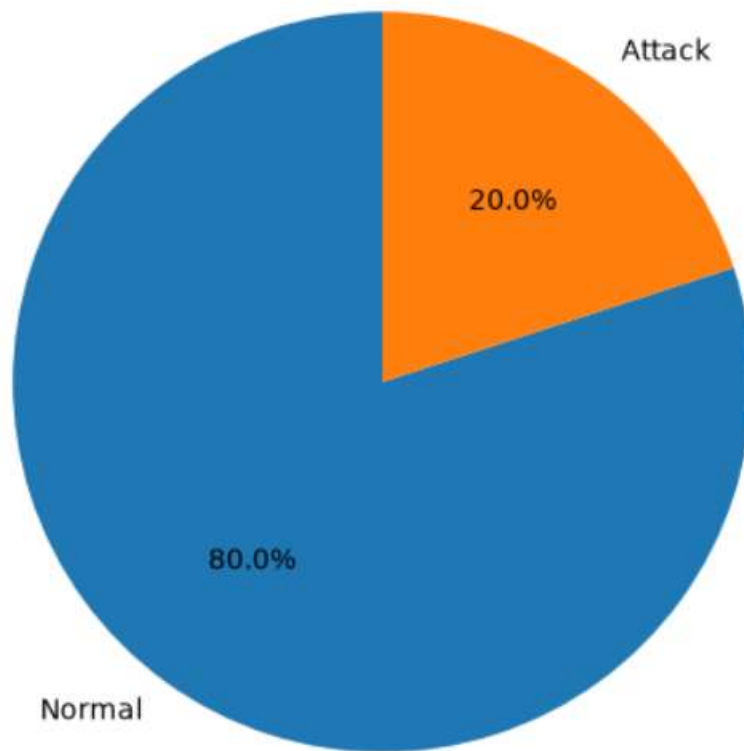
After training the Isolation Forest model, predictions were generated for all network connections.

The prediction distribution was visualized using a pie chart and a histogram.

- Observation
The majority of records were classified as normal, while the remaining records were identified as anomalous.
- Discussion

The contamination parameter controlled the expected percentage of anomalies, enabling the model to distinguish between normal and suspicious network activities.

Prediction Distribution



Prediction Pie Chart

7.10 Confusion Matrix Analysis

The confusion matrix compares the predicted labels with the actual labels.

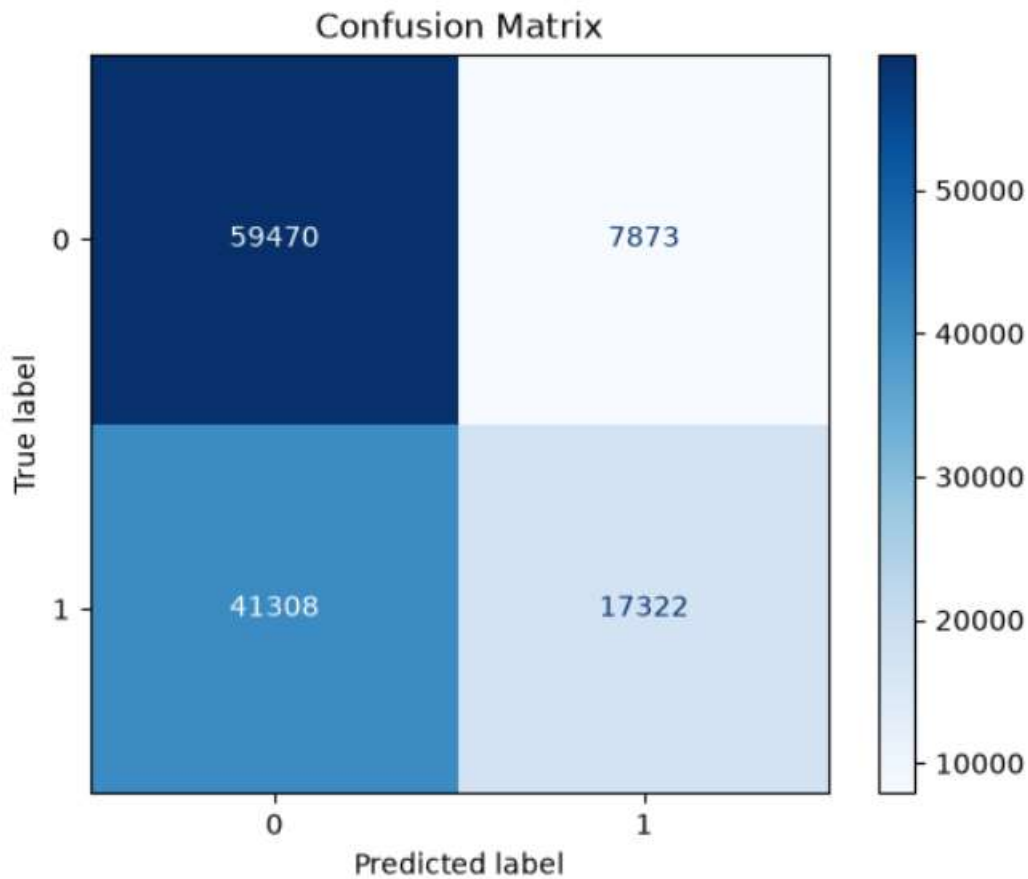
It consists of:

- True Positives (TP)
- True Negatives (TN)
- False Positives (FP)
- False Negatives (FN)

Discussion

The confusion matrix provides a detailed understanding of the classification performance. A larger number of correct predictions indicates that the model effectively identifies both normal and anomalous network traffic.

The confusion matrix also highlights misclassified records, which may occur because Isolation Forest performs anomaly detection without using labels during training.



7.11 Classification Report

The classification report evaluates the model using standard performance metrics.

The following metrics were calculated:

- Precision

Precision measures the proportion of predicted anomalies that are actually attacks.

Higher precision indicates fewer false alarms.

- Recall

Recall measures the ability of the model to identify actual attacks.

Higher recall indicates better detection capability.

- F1-score

The F1-score represents the harmonic mean of precision and recall.

It provides a balanced measure of model performance.

- Support

Support indicates the number of samples belonging to each class.

Discussion

The classification report demonstrates that the proposed Isolation Forest model successfully identified anomalous network traffic while maintaining reasonable prediction performance. Since the algorithm is unsupervised, slight differences between predicted and actual labels are expected.

```
print(classification_report(y, predictions))
```

	precision	recall	f1-score	support
0	0.59	0.88	0.71	67343
1	0.69	0.30	0.41	58630
accuracy			0.61	125973
macro avg	0.64	0.59	0.56	125973
weighted avg	0.64	0.61	0.57	125973

7.12 Overall Performance Discussion

The proposed AI-Based Network Intrusion Detection System successfully analyzed the NSL-KDD dataset and identified anomalous network activities using the Isolation Forest algorithm.

- The model demonstrated the following strengths:
- Efficient preprocessing of network data.

- Successful anomaly detection using unsupervised learning.
- Effective visualization of prediction results.
- Ability to identify suspicious network connections without using labeled data during training.
- The generated visualizations provided a clear understanding of network traffic behavior and model performance.

7.13 Advantages of the Proposed System

The proposed system offers several advantages:

- Detects unknown attacks using anomaly detection.
- Does not require labeled data during training.
- Fast execution using the Isolation Forest algorithm.
- Generates multiple graphical visualizations.
- Simple and easy to understand.
- Suitable for educational and research applications.

7.14 Limitations

Despite its advantages, the proposed system has certain limitations:

- Uses an offline dataset instead of real-time network traffic.
- Detection performance depends on dataset quality.
- Some modern cyberattack patterns are not included in the NSL-KDD dataset.
- Only one anomaly detection algorithm was implemented.

7.15 Chapter Summary

This chapter presented the experimental results of the proposed AI-Based Network Intrusion Detection System. Various analyses, including missing value checks, protocol distribution, byte distributions, prediction distribution, confusion matrix, and classification report, demonstrated the effectiveness of the Isolation Forest algorithm in identifying anomalous network traffic. The results indicate that the

proposed system provides a practical and efficient approach for network intrusion detection using unsupervised machine learning.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

In this project, an AI-Based Network Intrusion Detection System was successfully developed using the Isolation Forest algorithm and the NSL-KDD dataset. The primary objective of the project was to detect abnormal network activities using an unsupervised machine learning approach without relying on labeled data during model training.

The implementation began with loading and exploring the NSL-KDD dataset. Various preprocessing techniques, including duplicate checking, missing value analysis, label encoding, and feature scaling, were performed to prepare the dataset for machine learning. These preprocessing steps improved the quality of the input data and ensured compatibility with the Isolation Forest algorithm.

The Isolation Forest model was then trained to identify anomalous network connections. The model successfully distinguished between normal and suspicious network traffic by isolating unusual observations in the dataset. The predicted results were evaluated using a confusion matrix and a classification report. Various visualizations, including prediction distribution, protocol distribution, actual label distribution, source bytes distribution, and destination bytes distribution, were generated to better understand the model's behavior and performance.

The results demonstrate that the proposed system can effectively detect anomalous network traffic while requiring minimal human intervention. Since the model is based on unsupervised learning, it has the potential to identify previously unseen attacks that may not be detected by traditional signature-based intrusion detection systems.

Overall, the project successfully achieved its objectives and demonstrated the practical application of artificial intelligence in enhancing network security.

8.2 Achievements of the Project

The following objectives were successfully achieved during the implementation:

- Developed an AI-based Network Intrusion Detection System.
- Successfully loaded and analyzed the NSL-KDD dataset.
- Performed data preprocessing and feature engineering.
- Applied Label Encoding and Feature Scaling.

- Implemented the Isolation Forest algorithm.
- Detected anomalous network traffic.
- Evaluated the model using a confusion matrix and classification report.
- Generated multiple visualizations for result analysis.
- Saved the trained machine learning model for future use.

8.3 Applications

The proposed intrusion detection system can be applied in several areas, including:

- Enterprise network security
- Educational institutions
- Banking and financial organizations
- Government departments
- Cloud computing environments
- Data centers
- Cybersecurity research
- Security Operations Centers (SOC)

8.4 Future Scope

- Although the proposed system performs effectively on the NSL-KDD dataset, several improvements can be made in future work.
- Real-Time Intrusion Detection
- Instead of analyzing an offline dataset, the model can be integrated with live network traffic to provide real-time intrusion detection.

- Comparison with Multiple Algorithms
- The performance of the proposed system can be compared with other anomaly detection algorithms such as:
 - DBSCAN
 - Local Outlier Factor (LOF)
 - One-Class SVM
 - Autoencoders

This would provide a broader understanding of different unsupervised learning techniques.

- Deep Learning-Based Intrusion Detection

Deep learning models such as Autoencoders or LSTM networks can be used to improve anomaly detection performance for complex network traffic patterns.

- Cloud-Based Deployment.

The intrusion detection system can be deployed on cloud platforms to monitor large-scale distributed networks.

- IoT Network Security

The proposed methodology can be adapted for Internet of Things (IoT) environments, where billions of connected devices generate network traffic.

- Web-Based Dashboard

A user-friendly web application using Streamlit or Flask can be developed to allow users to upload network traffic data and obtain intrusion detection results interactively.

- Explainable Artificial Intelligence (XAI)

Future systems can incorporate Explainable AI techniques to help security analysts understand why the model classified a network connection as anomalous.

8.5 Final Summary

The project successfully demonstrated that machine learning can be effectively used to detect network intrusions. By employing the Isolation Forest algorithm, the system identified anomalous network activities without requiring labeled training data. The combination of proper data preprocessing, anomaly detection, performance evaluation, and visualization resulted in a simple yet effective intrusion detection system.

The project provides a strong foundation for future research in cybersecurity and demonstrates how artificial intelligence can contribute to improving network security.

REFERENCES

Use the following references in your report:

1. Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A Detailed Analysis of the KDD CUP 99 Data Set. IEEE Symposium on Computational Intelligence for Security and Defense Applications.

2. Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. Proceedings of the IEEE International Conference on Data Mining (ICDM).
3. Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow. O'Reilly Media.
4. Python Software Foundation. Python Documentation.
5. Scikit-learn Developers. Scikit-learn User Guide.
6. McKinney, W. Python for Data Analysis. O'Reilly Media.
7. NumPy Developers. NumPy Documentation.
8. Matplotlib Development Team. Matplotlib Documentation.
9. Pandas Development Team. Pandas Documentation.
10. NSL-KDD Dataset Documentation.